

Distributed Tic-Tac-Toe: Technical Report

Alexandru Eduard Toni

September 21, 2026

1 Analysis & Requirements

The goal of this project is to implement a distributed, concurrent multi-player Tic-Tac-Toe game using **Java RMI (Remote Method Invocation)** for network communication and **JavaFX** for the graphical user interface.

- **Match Management:** Players can dynamically create a new match by specifying a unique match name or join an existing one.
- **Turn-Based Concurrency:** The system must handle multiple concurrent games safely, ensuring players take alternating turns.
- **Real-time Notifications:** The server must broadcast game events (start, moves, game over) asynchronously to all participants in a match using RMI callbacks.

2 System Architecture & Design

The system follows a distributed client-server architecture using the **Remote Procedure Call (RPC)** paradigm combined with the **Observer Pattern**.

2.1 Class Structure (UML Overview)

- **GameEngine (Interface):** Remote interface exposed by the server containing `joinGame()` and `makeMove()`.
- **GameEngineImpl:** Server-side implementation managing active boards, players, and turns.
- **GameClientListener (Interface):** Remote callback interface residing on the client to receive asynchronous updates (`notifyMoveMade`, `notifyGameOver`, `notifyGameStarted`).
- **GameController:** JavaFX frontend controller handling user input, lobby navigation, and board updates.

3 Interaction Flow

The lifecycle of a game session follows these distributed steps:

1. **Connection & Lobby:** The client starts, looks up the "GameEngine" in the RMI registry (`localhost:1099`), and invokes `joinGame(Message, listener)`.
2. **Game Initialization:**

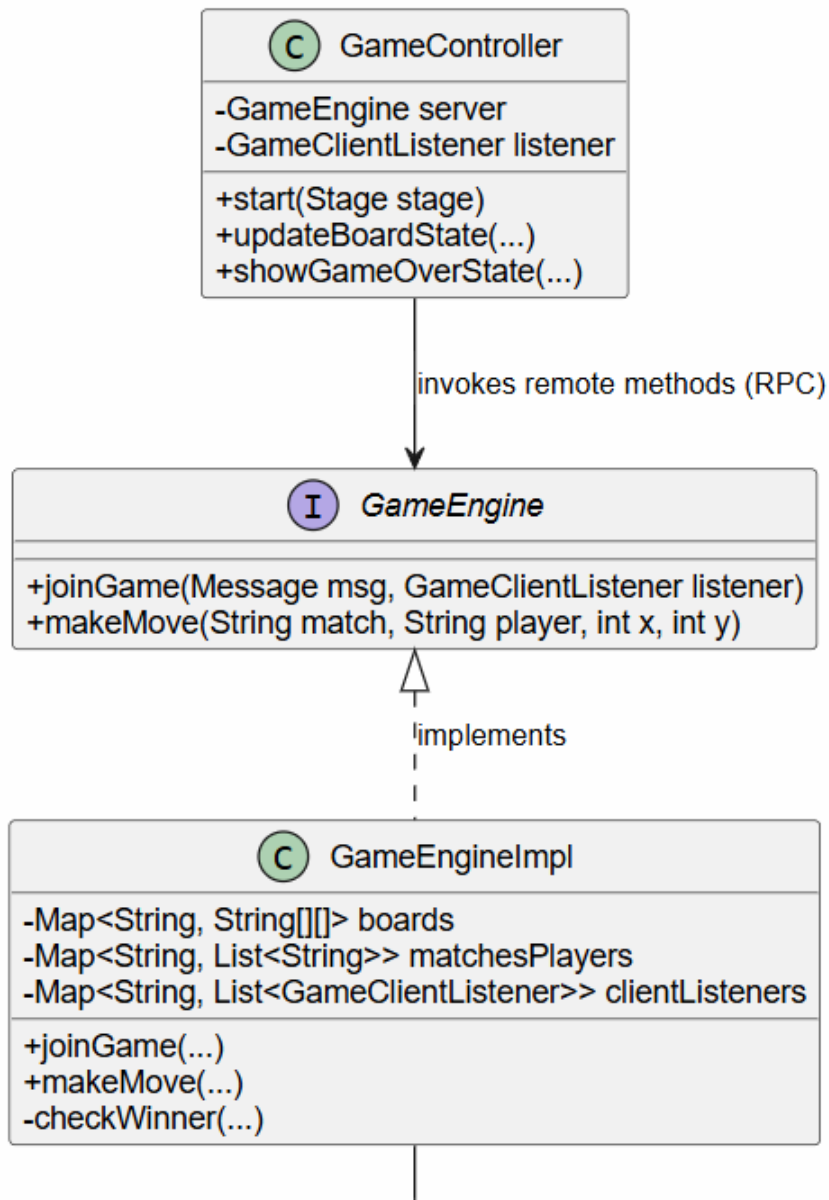


Figure 1: Project class diagram 1.

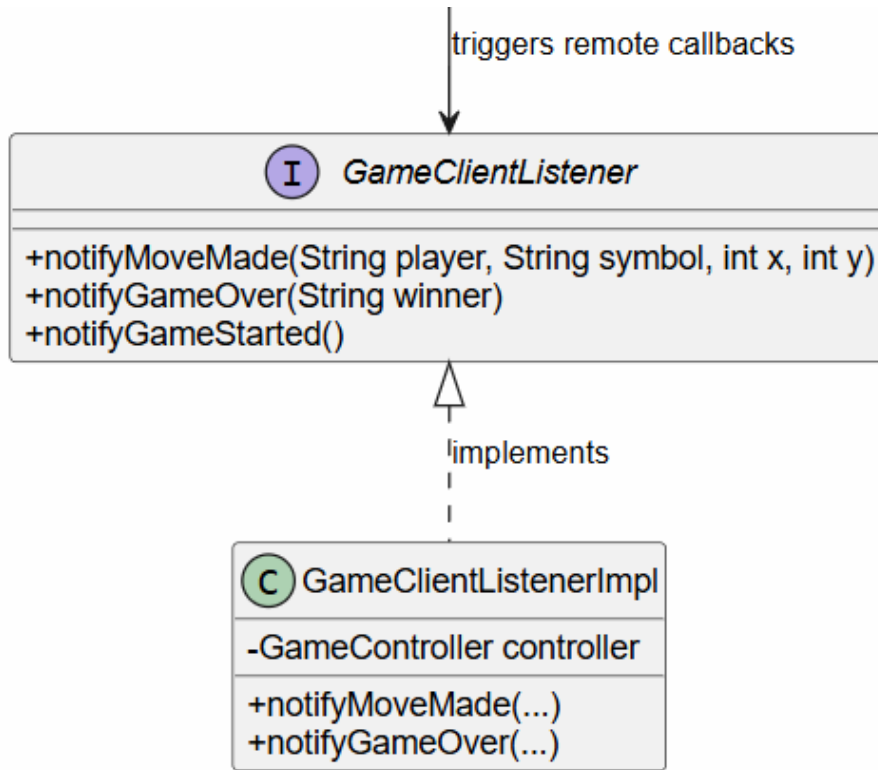


Figure 2: Project class diagram 2.

- If the match does not exist, the server initializes a 3×3 board.
- When the second player joins, the server automatically triggers `notifyGameStarted()` via remote callbacks.

3. Move Execution Loop:

- A player clicks a cell; the client invokes `server.makeMove(...)`.
 - The server validates turns and boundaries inside a thread-safe block, updates the board, and broadcasts `notifyMoveMade(...)` to all match listeners.
4. **Termination:** The server checks for win/draw conditions, broadcasts `notifyGameOver(...)`, locks the UI, and clears the match from memory.

4 Implementation Highlights

- **Thread Safety on Server:** Utilizes `ConcurrentHashMap` and `CopyOnWriteArrayList` for tracking room states safely, alongside explicit `synchronized (board)` blocks to prevent race conditions during moves.
- **UI Thread Concurrency:** Remote callback executions are dispatched safely to the JavaFX application thread using `Platform.runLater(...)` to avoid multi-threading UI exceptions.